CS 252:

Advanced Programming Language Principles

The (Untyped) Lambda Calculus

Prof. Tom Austin
San José State University

Minimum complete programming language?

WARNING: I expect you to remember every construct of this language for exams

Lambda Calculus expressions

`e ::=`	*expressions:*
`x`	variables
`| (λx.e)`	lambda abstractions
`| e e`	function application

We could have just said "function", but we want to sound cool

Lambda Calculus values

`v ::=` *values:*

`(λx.e)` lambda abstractions

When our program finishes running, it returns some complex function as its "value"

Function application

Suppose we have a function:

`(λx.E)`

Where E is some complex expression.

How do we evaluate:

`(λx.E) v` → `E[x->v]`

v replaces x wherever it occurs in E

Small step semantics for λ-calculus

(in-class)

Operational Semantics

[Ctxt1]
$$\frac{\texttt{e1 -> e1'}}{\texttt{e1 e2 -> e1' e2}}$$

[Ctxt2]
$$\frac{\texttt{e2 -> e2'}}{\texttt{(}\lambda\texttt{x.e) e2 -> (}\lambda\texttt{x.e) e2'}}$$

[Call]
$$\texttt{(}\lambda\texttt{x.e) v -> e[x->v]}$$

Example: Identity Function

```
   (λx.x) (λa.λb.a)
→  x[x->(λa.λb.a)]
→  (λa.λb.a)
```

When should we evaluate function arguments?

Strict Evaluation Strategies

Evaluate function arguments first

- *Call-by-value:*
  copy of the parameter is passed
- *Call-by-reference:*
  implicit reference is passed

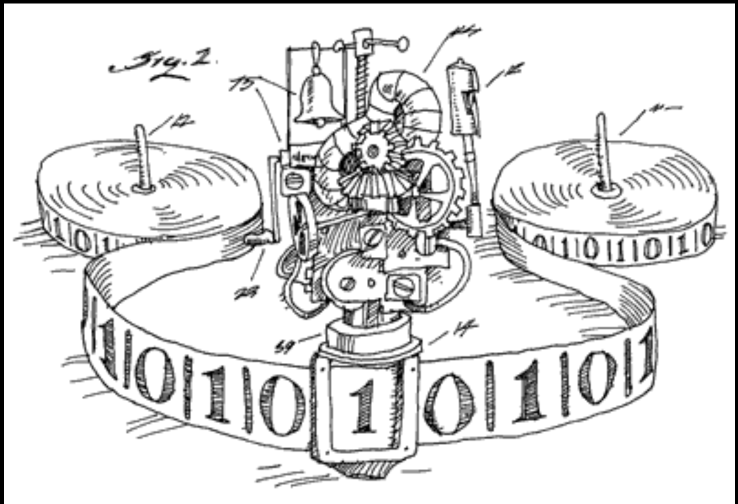
Lazy Evaluation Strategies

Substitute arguments in function body

- *Call-by-name:*
  re-evaluate the argument each time
- *Call-by-need:*
  memoizes parameter value after use

How powerful is this language?

The lambda-calculus is *Turing complete*.



You can also implement the λ-calculus w/ a Turing machine

In other words, the two are **equal in power**

Translating λ-calc to Haskell

Lambda-calculus	Haskell
`x`	`x`
`(λx.e)`	`(\x -> e)`
`e e`	`e e`

Extending the lambda calculus

(in class)

Test function

Imagine an if/then/else like `test` function:

```
test = λcond.λthn.λels.cond thn els
```

For this to work, how would `true` and `false` need to be defined?

*(Remember — true and false must be **functions**.)*

Boolean Values

- `true = λx.λy.x`
  - Get first argument
- `false = λx.λy.y`
  - Get second argument
- `test = λcond.λthn.λels.cond thn els`

In class: work through both

1. `(test true true false)`
2. `(test false true false)`

Lab: Define `and`, `or`, and `not` for the λ-calculus

- The `and` case is done for you

Pairs

- `pair = λf.λs.(λb.b f s)`
  - ‘b’ is a boolean.
  - Remember: ‘true’ and ‘false’ act as ‘get first arg’ and ‘get second arg’, respectively.
- `first = λp.p true`
- `second = λp.p false`

Lists

- Lists are treated as special pairs:
  - `cons = pair`
  - `head = first`
  - `tail = second`
- `isEmpty` exploits structure of pairs
  - returns false as long as the argument is a pair
  - need to define `nil` carefully.

`isEmpty` example, one-element list

```
    isEmpty (λb.b true nil)
=   (λlst.lst (λh.λt.false))
                (λb. b true nil)
→   (λb. b true nil) (λh.λt.false)
→   (λh.λt.false) true nil
→   (λt.false) nil
→   false
```

`isEmpty` example, empty list

```
    isEmpty nil
=   (λlst.lst (λh.λt.false)) nil
→   nil (λh.λt.false)
=   ??? (λh.λt.false)
→   true
```

hint: whatever `nil` is given, it should evaluate to `true`.

Church Numerals

(AKA do something N times)

- Essentially, instructions for producing numbers
- Inputs:
  - `z`: Representation of zero
  - `s`: Successor function that works with `z`
- Examples:
  - `zero  = λs.λz.z`
  - `one   = λs.λz.s z`
  - `two   = λs.λz.s (s z)`
  - `three = λs.λz.s (s (s z))`

Functions for Numbers

- `succ = λn.(λs.λz.s (n s z))`
  - `n s z` – "apply `s` to `z` `n` times"

- `plus = λm.λn.(λs.λz.m s (n s z))`

- `plus' = λm.λn.m succ n`
  - Alternate definition of `plus` that uses `succ`.
  - "Apply `succ` to `n` `m` times."

Lab: Define `multiply` for the λ-calculus

- Consider function currying — you may need a "plus n" function.

Omega (Ω)

```
      (λx.x x) (λx.x x)
→     x x[x->(λx.x x)]
→     (λx.x x) (λx.x x)
→     x x[x->(λx.x x)]
→     (λx.x x) (λx.x x)
→     …
```

Y Combinator

- Similar to omega
- Also called the ‘fix’ combinator
- Produces recursive functions
- `fix = λf.(λx.f (λy.x x y)) (λx.f (λy.x x y))`

Lab: Develop new features in the Lambda Calculus using Haskell

Details on Canvas.

Starter code is available on the course website.